

OBJECT ORIENTED PRINCIPLES

ASSIGNMENT 3: A JAVA APPLICATION FOR MANAGING A STORE.

Link to Youtube - https://youtu.be/Au3_jWEDMoo

Declaration of Authorship

I, Rebecca, declare that the work presented in this assignment titled 'A Java Application for Managing a Store' is my own. I confirm that:

- This work was done wholly by me as part of my BSc. (Hons) in Software Development, my Msc at Munster Technological University.
- Where I have consulted the published work and source code of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this assignment source code and report is entirely my own work.

On 10th December 2024

Signature: Rebecca

Java Application Description.

This java application is designed to manage a bakery through an interactive, text-based menu system. The functionality of the application includes:

- Adding bakery products such as cakes, pasties and bread.
- Placing orders and viewing the details of those orders.
- Viewing a list of bakery products and checking order statuses.

This interactive system helps manage a bakery by keeping track of products, customer orders and ensuring the bakery's information is up to date.

Technical Difficulty: OOP Concepts Demonstrated in the Java Application.

1. Primitive and Reference Variables:

In the Item class, price of a bakery item is a primitive double, while the order that tracks a customer's purchase is a reference variable pointing to an Order object.

2. Class and Objects

The system uses classes to represent real-world entities like bakery items (Item, Cake, Bread, Pastry), customers and orders. These classes define the properties and behaviours for each object in the bakery.

3. Encapsulation

To keep data safe, we use encapsulation. In the Item class, fields like name and price are private. We use public methods like getName() and setName() to access and update these fields.

4. Aggregation

The Oder Class aggregates a list of Item objects, showing how an orders is made up of different products. The Customer class aggregates multiple Order objects, allowing customers to have multiple orders over time.

5. Inheritance

The classes Cake, Bread and Pastry inherit from the Item class. This allows them to share common properties like name and price but also have their own specific features like flavour or filling.

6. Class Hierarchy

The Cake, Bread and Pastry classes are subclasses of the Item class. These subclasses inherit common properties and methods from Item, such as getPrice(), but also introduce their own unique features.

7. Static Polymorphism(overloading)

The method `addItem()` in the `BakeryImp` class is overloaded to handle different types of products such as cakes, bread and pastries, based on the parameters passed to it.

8. Dynamic Polymorphism(overriding)

The `toString()` method is overridden in `Cake`, `Bread` and `Pastry` classes to provide specific details about each type of item. When the `toString()` method is called on any `Item`, it returns the correct description of that item.

9. Abstract Class

The `Item` class is abstract which means it can't be directly used to create objects. Subclasses like `Cake`, `Bread` and `Pastry` inherit from `Item` and provide their own specific details.

10. Interface

The `Bakery` interface defines essential actions for managing bakery items, like adding products. The `BakeryImp` class implements this interface, handling the actual functionality behind the scenes.

11. User and Developer Isolation

The `MyMain` class interacts with the `Bakery` interface to use bakery functionality without needing to understand the details of its implementation. This makes the system easier to maintain.

12. Upcasting

In the `MyMain` class upcasting is used when assigning a `Cake` or `Bread` object to an `Item` reference, allowing uniform handling of all product types in the bakery.

13. Static Fields and Methods

Static fields, like `nextId` in the `BakeryImp` class are shared by all instances of the class, not just one specific object. Static methods like `getItemCount()`, can be called without creating an object.

14. Final Fields, Methods and Classes

The `BakeryImp` class is marked as `final` to prevent inheritance, ensuring that the class cannot be extended and modified by subclasses.

15. Data Structures

The bakery uses an `ArrayList<Item>` to store the bakery's products and an `ArrayList<Order>` to store customer orders. These data structures make it easy to add, remove and manage items and orders.

16. Java Generics

The `ArrayList<Item>` and `ArrayList<Order>` use of Java Generics, which ensure that only `Item` or `Order` objects can be stored, promoting type safety.

17. Downcasting

Downcasting occurs when accessing specific methods from the Cake, Bread, or Pastry subclasses, stored in an ArrayList<Item>. This allows the program to access subclass specific features.

18. Exception Handling

Exception Handling ensure the program doesn't crash when the user enters invalid data. E.g. If a user enters a non-integer value where a number is expected, the program catches the error and prompts the user to try again.

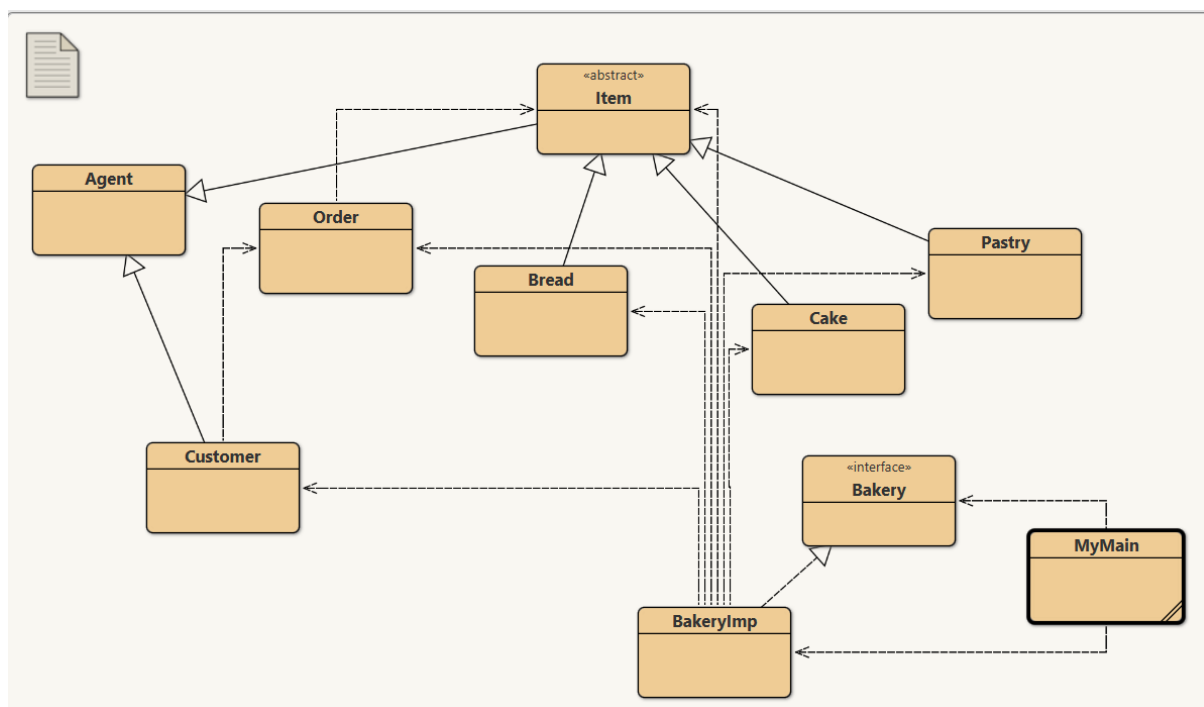
19. File Reading and Writing

The BakeryImp class implements methods for saving and loading data, such as items and orders, from a file. The ensures the bakery's information stays safe.

20. Default Constructor and Copy Constructor

The Item class includes a default constructor that initialises default values. It also includes a copy constructor for duplicating objects when needed.

UML Design: Java Application.



Testing the Java Application.

The functionality of the bakery application is tested in MyMain.java through an interactive, text-based menu. The user can select options to add bakery items such as cake, pasties and bread, place orders and view order details. For example, when adding an cake, the user is prompted for details like the cakes name, flavour and price, which are then saved in the system. Similarly, when placing an order, the user selects multiple items, and the system creates an order with those items.